

Automated LZW Exercise Generation

Generating fair, unique assessment questions for an algorithmics course

Andrew King

The Problem

- Giving every student the same paper creates potential collusion risk
- Composing unique assessment papers manually is time-consuming and hard to calibrate fairly
- The challenge: questions need to be varied enough to prevent collusion, but similar enough in difficulty to be fair

LZW algorithm

- Dictionary-based compression algorithm
- Initialise dictionary with base alphabet: A=00, C=01, G=10, T=11
- Step-by-step execution maps naturally onto an exam table

LZW compression – Example

Text = G A C G A T A C G A T A C G
File size = 14 bytes, or 28 bits if 2 bits/char

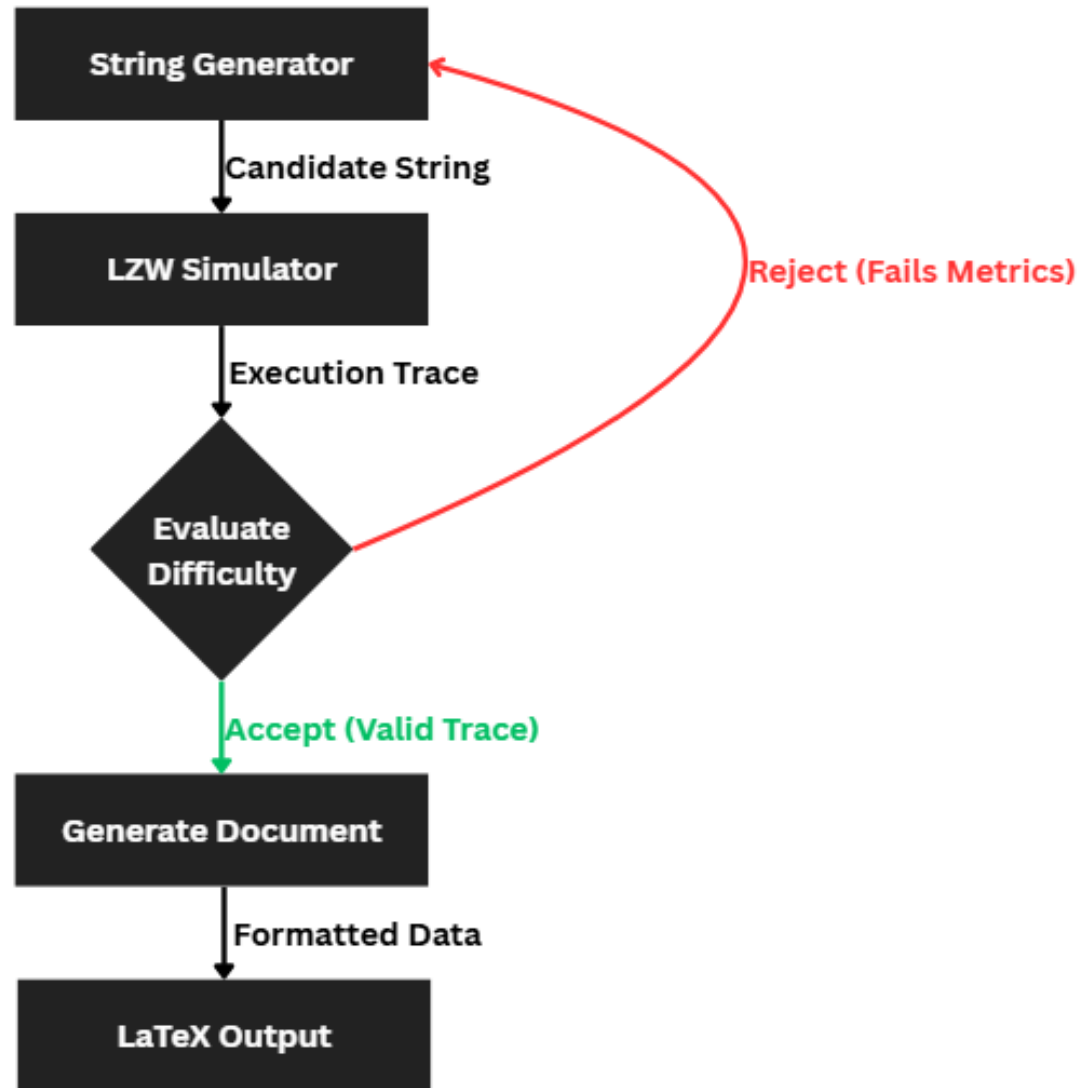
Compressed file: 10000001100011010101111001
file size = 26 bits

step	position in string	longest string in dictionary	b	add to dictionary	code
1	1	G	10	GA	4
2	2	A	000	AC	5
3	3	C	001	CG	6
4	4	GA	100	GAT	7
5	6	T	011	TA	8
6	7	AC	0101	ACG	9
7	9	GAT	0111	GATA	10
8	12	ACG	1001	-	-

Controlling Difficulty

- **Quantitatively** step count and dictionary growth
- **Statistically** Shannon entropy captures character-level predictability $H = - \sum p(x) \times \log_2(p(x))$
- **Perceptually** how patterned does the string *look* to a student?

How the Generator Works



- Generate-and-filter loop: sample → streak test → simulate → check targets → accept or retry
- Up to 100,000 attempts before fallback

Output

- Two exercise types: compression and decompression
- Two difficulty profiles: high entropy and low entropy
- LaTeX output to slot into existing class test paper workflow
- First row pre-filled as worked example
- Extra blank rows conceal true step count

LZW Compression Question

Input Sequence:

T	C	A	T	T	C	T	C	T	A	T	G
---	---	---	---	---	---	---	---	---	---	---	---

Using the LZW algorithm, compress the sequence from left to right.

Start with the initial dictionary: A = 00, C = 01, G = 10, and T = 11 (Indices 0–3).

At each step, identify the longest matching string in the dictionary and output its binary code. Form a new dictionary entry by appending the next symbol in the sequence to the current match, assigning it the next available integer index. Complete the table provided below.

Step	Position in String	Longest String in Dictionary	k	Binary Encoding	String Added	Code Created
1	1	T	2	11	TC	4

LZW Compression Question

Input Sequence:

T	C	A	T	T	C	T	C	T	A	T	G
---	---	---	---	---	---	---	---	---	---	---	---

Using the LZW algorithm, compress the sequence from left to right.

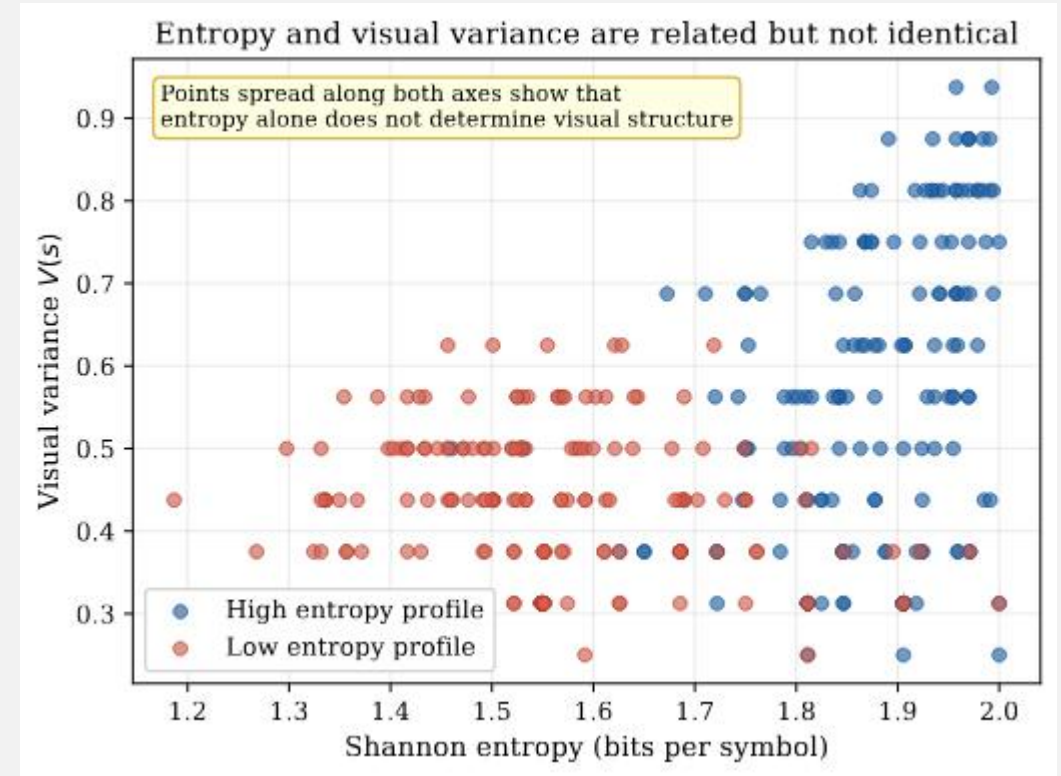
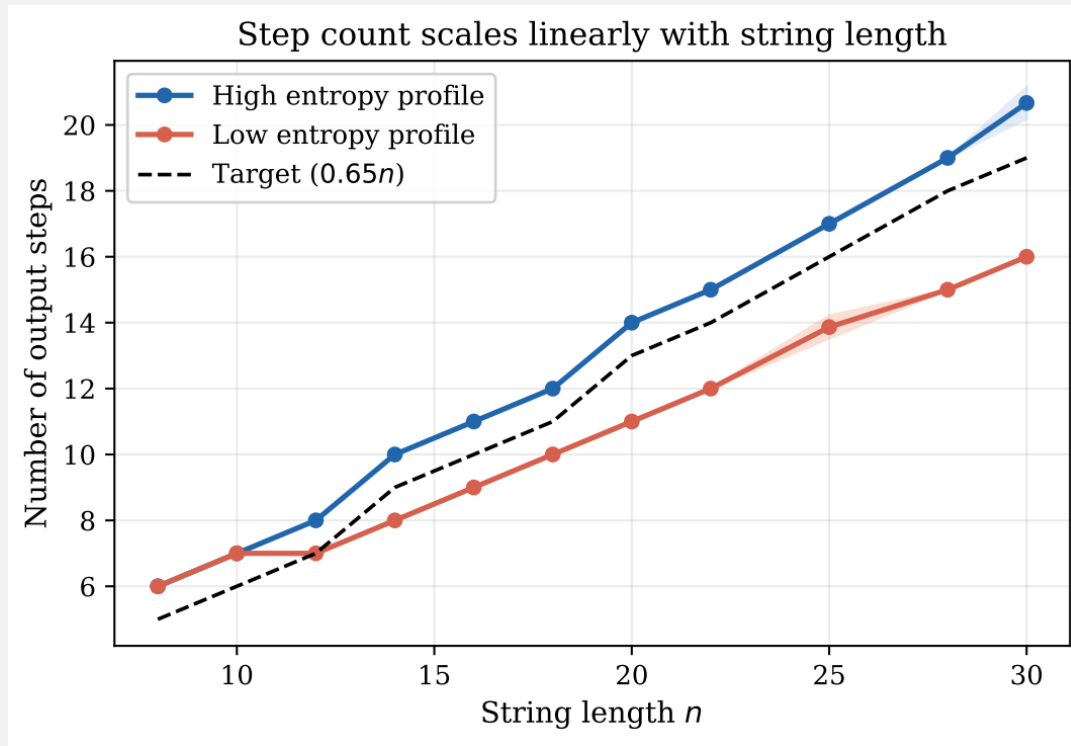
Start with the initial dictionary: A = 00, C = 01, G = 10, and T = 11 (Indices 0–3).

At each step, identify the longest matching string in the dictionary and output its binary code. Form a new dictionary entry by appending the next symbol in the sequence to the current match, assigning it the next available integer index. Complete the table provided below.

Examiner Solution Key

Step	Position in String	Longest String in Dictionary	k	Binary Encoding	String Added	Code Created
1	1	T	2	11	TC	4
2	2	C	3	001	CA	5
3	3	A	3	000	AT	6
4	4	T	3	011	TT	7
5	5	TC	3	100	TCT	8
6	7	TCT	4	1000	TCTA	9
7	10	AT	4	0110	ATG	10
8	12	G	4	0010	-	-

Evaluating the Solution - Quantitative



Evaluating the Solution - Qualitative

The question generator was tested the following ways:

- Peer usability test confirmed that the question generator was intuitive to use and that difficulty metrics led to fairer difficulty generation
- Algorithms course class test conducted using generated questions. Results from student post-test survey shown below:

Question	Positive	Other
Confidence in LZW	5/5 (100%)	0
Similarity to class examples	4/5 (80%)	1 neutral
Tested understanding	5/5 (100%)	0
Clarity of generated input	5/5 (100%)	0
Fairness of differentiation	2/5 (40%)	1 neutral, 2 negative

Outcomes & Possible Future Developments

Achieved Outcomes

- Generates unique LZW compression and decompression exercises
- LaTeX output fit for use in class tests
- Students rated exercises as clear, fair, and comparable to class examples

Possible Future Developments

- Huffman coding support: similar table-based structure, where the approach can be adapted
- Student practice tool to generate personalised revision exercises